

The Mathematics of Deep Tech for Artificial Intelligence

A Problem-First Curriculum for Frontier Technology

Why Deep Tech is best understood not as a collection of unrelated fields, but as a small set of recurring mathematical structures — and why mathematics should therefore be taught through Deep Tech rather than before it.

[Ajit Jaokar](#)

Erdos Research

AI Research & Education

Version 1.0 · June 2026

Contents

Introduction	2
Background	2
Meaning of a Learned Surrogate	3
1. The Leitmotif of Mathematics for Deep Tech	4
2. The Inversion: Deep Tech Problem → Mathematical Lens	4
3. Seven Themes of Deep Tech Mathematics	5
3.1 Representation — how do we represent reality?	5
3.2 Dynamics — how does reality evolve?	5
3.3 Uncertainty — what don't we know?	6
3.4 Optimisation — what is the best solution?	6
3.5 Learning — what if we cannot write the equations?	6
3.6 Control — how do we make systems behave?	6
3.7 Computation — what can actually be computed?	6
Summary of the seven themes	6
4. From Themes to Verbs	7
5. The Modelling Stack: Reality → Equation → Simulation → Surrogate	8
6. Proposed Curriculum: Ten Modules	9
Module 1 — How We Model Reality	9
Module 2 — How Systems Change	9
Module 3 — How We Simulate Nature	9
Module 4 — How We Reason Under Uncertainty	9
Module 5 — How We Optimise	9
Module 6 — How We Learn Models	10
Module 7 — How We Learn Physics	10
Module 8 — How We Control Complex Systems	10
Module 9 — How Information Flows	10
Module 10 — How Nature Computes	10
How the modules map to themes and verbs	10
7. Pedagogical Principles	11
8. Expansion of the idea of the Learned Surrogate	11
9. Conclusion	13

Introduction

Background

Mathematics is almost always taught discipline by discipline — linear algebra, then calculus, then probability, then optimisation, then differential equations — with applications postponed until the maths foundation is complete.

For students and practitioners whose real interest is frontier technology, this paper argues that the ordering should be inverted.

Deep Tech can be viewed as a small set of recurring mathematical structures applied in different contexts.

A fusion reactor, a drug-discovery pipeline and an autonomous vehicle differ enormously in their physics, but their mathematical skeletons have many similarities: each *represents* a high-dimensional state, models how that state *evolves*, reasons under *uncertainty*, *optimises* an objective, and increasingly replaces expensive computation with *learned surrogates*.

Of course the surrogate framing is not the only approach possible to model Deep Tech - but it is an approach that covers many important aspects and is a tangible starting point.

This paper primarily focuses on AI Education for Deep Tech using a maths framework both at the University of Oxford and at [Erdos Research](#). It is developed in my teaching and will be largely shared openly. The views shared are my own (Ajit Jaokar).

The paper proposes organising mathematical education around seven themes — Representation, Dynamics, Uncertainty, Optimisation, Learning, Control and Computation — and seven corresponding operations, which we can call the mathematical verbs of Deep Tech: Represent, Simulate, Infer, Optimise, Learn, Control and Compute.

The framing is oriented towards the teaching of **Artificial Intelligence for Deep Tech - a Mathematics foundation**

The central claim is that every Deep Tech company is some combination of these verbs. This framing makes the mathematics legible and motivated, and it accommodates the most consequential recent shift in scientific practice: the move from Reality → Equation → Simulation to Reality → Equation → Simulation → Learned Surrogate. The remainder of the paper develops the argument and then sets out a ten-module curriculum that embodies it.

I am thankful to [Prof Dean Mohamedally](#) and [Dr Simon Banks](#)

By sharing our work, I hope to nurture an ecosystem for Deep Tech and AI Education, especially in the UK. We, at the University of Oxford are also planning a course on Artificial Intelligence for Deep Tech.

In a previous post, I shared [context and definitions of maths for Deep Tech](#) which acts as a background for this paper

This work (Artificial Intelligence for Deep Tech) is part of a broader approach I am taking for developing the **Mathematical foundations of Artificial Intelligence** (for teaching).

Meaning of a Learned Surrogate

In this context, we define a surrogate as follows:

A surrogate is a computational model that stands in for a more expensive, inaccessible, complex, or unknown process while preserving the behaviour that matters for a given task.

This definition is deliberately broader than the traditional engineering definition. In engineering and scientific computing, a surrogate is: A learned approximation of an expensive simulation.

For example: Weather surrogates, Materials discovery models etc. This is the definition most people use today. They are synonymous with the word ‘simulation’ but mathematically, they relate to cases where [closed form solutions](#) do not exist for complex systems.

We can ask a broader question: *What is actually being replaced?*

The answer is not always a simulation.

Sometimes it is:

- a physical process
- a mathematical model
- a search process
- a planning process
- a human decision process

1. The Leitmotif of Mathematics for Deep Tech

I like the German work **leitmotif** - which broadly means a short, recurring phrase or idea - originally applied in the musical sense

We can think of mathematics in this context (as a recurring set of ideas) across Deep Tech.

The current, Discipline-first teaching, starting with Linear Algebra etc obscures the most useful fact about applied mathematics: that the same structure recurs across domains. A student who meets eigenvalues in a linear algebra course, stability analysis in a differential equations course, resonance in a physics course and principal components in a statistics course is rarely shown that these are, at heart, one idea encountered four times. Each discipline is taught in its own individual silo, and the transferable insight — that frontier technology is built from a handful of repeating mathematical moves — is precisely what the curriculum's structure obfuscates.

The current default sequence can be summarised as **Math Topic** → **Application**. The mathematics is established first and the application is offered afterwards as illustration. There is some value in inverting the direction of this arrow as we shall see below.

2. The Inversion: Deep Tech Problem → Mathematical Lens

We can reverse the arrow. Instead of **Math Topic** → **Application**, the organising unit becomes **Deep Tech Problem** → **Mathematical Lens**. The problem-first approach shows the why (demand for the abstraction) and then addresses the abstraction maths underlying the demand).

For Example, Confinement instability motivates dynamical systems; noisy diagnostics motivate Bayesian inference; an intractable simulation motivates a learned surrogate. The abstraction is retained because it was related to a problem the learner already cared about.

This is a ‘sequencing of learning’ claim from a pedagogical perspective and should not impact the rigour. A reasonable objection against this approach is that mathematics has a real dependency structure — one cannot do partial differential equations without calculus, nor principled inference without probability — and that a problem-first ordering risks teaching technique without foundation.

While this dependency is valid - it need not be rigid - especially if we adopt a more spiral approach to learning which we aspire to ([Spiral Curriculum - Jerome Bruner](#) - wherein fundamental mathematical concepts would be revisited repeatedly throughout a course and progressively expanded to greater levels of complexity.

The inversion also reframes what a curriculum is for. Its job is no longer to deliver a complete tour of mathematics, but to give a practitioner the smallest set of structures that lets them read, build and reason about frontier systems. That set is intentionally compact - even if it expands progressively.

Finally, today, AI assisted learning helps you speed up the understanding of many foundational ideas - making the higher level application of these ideas more valuable. In addition, our emphasis is relatively narrow (Deep Tech) - please see my previous post for this background [context and definitions of maths for Deep Tech](#)

3. Seven Themes of Deep Tech Mathematics

If we consider the major frontier technologies — fusion, robotics, drug discovery, artificial intelligence, quantum, climate, semiconductors — a set of seven questions recur. We treat these as the conceptual axes of the field. Each names a question that reality poses, the body of mathematics that answers it, and the domains in which it applies.

3.1 Representation — how do we represent reality?

Before anything can be computed, the system must be encoded as a mathematical object. Vectors, tensors, manifolds, graphs and latent spaces are the vocabulary of representation. The choice of representation is rarely neutral: it determines which operations are cheap, which symmetries are visible, and which problems become tractable. Molecules become graphs, quantum systems become state vectors, a robot becomes a point in configuration space, and an organisation's knowledge becomes a graph of entities and relations.

3.2 Dynamics — how does reality evolve?

Most frontier systems are not static; they change in time. Ordinary and partial differential equations, dynamical systems, chaos and stochastic processes describe how a state moves. Stability, sensitivity to initial conditions and the existence of attractors are the questions that matter here. Weather, fluids, plasma in a fusion device and the dynamics of populations all live in this theme.

3.3 Uncertainty — what don't we know?

Real measurements are noisy and real knowledge is incomplete. Probability, statistics, Bayesian methods and information theory turn that ignorance into something a system can reason about. The key shift, made explicit in the curriculum below, is to treat uncertainty as a first-class engineering object rather than an inconvenience: a sensor reading is a distribution, not a number. Sensing, scientific experimentation, climate forecasting and robotic state estimation all depend on this theme.

3.4 Optimisation — what is the best solution?

A vast amount of engineering reduces to searching a space of designs or decisions for the best one. Convex optimisation, combinatorial optimisation and variational methods provide the machinery. Whether a problem is convex or non-convex, continuous or discrete, often matters more to the engineer than the domain it came from. Chip layout, antenna design, protein folding and logistics are optimisation problems in different clothing.

3.5 Learning — what if we cannot write the equations?

Sometimes no tractable governing equation exists, or the one that does is too expensive to solve. Machine learning, deep learning, foundation models and surrogate models approximate the unknown function directly from data. The conceptual core is simple: learning is function approximation under constraints. AlphaFold, large language models and data-driven materials discovery are all instances of this theme.

3.6 Control — how do we make systems behave?

Representing, simulating and predicting a system is not the same as making it do what we want. Control theory, feedback and reinforcement learning close the loop between observation and action. Observability and controllability — what can be inferred and what can be steered — are the structural questions. Robotics, autonomous vehicles, power grids and manufacturing lines are all problems in control.

3.7 Computation — what can actually be computed?

Finally, every method must run on real hardware in finite time. Complexity theory, numerical methods, high-performance computing and quantum computing describe what is feasible and at what cost. Computation sets the boundary conditions on everything else: a model that cannot be evaluated in time is no model at all. Simulation, cryptography and quantum algorithms live here, and so, increasingly, does the question of whether physics itself can be made to compute.

Summary of the seven themes

Theme	Mathematics	Representative domains
Representation	Vectors, tensors, manifolds, graphs, latent spaces	Molecules, quantum states, robot states, knowledge graphs
Dynamics	ODEs, PDEs, dynamical systems, chaos, stochastic processes	Weather, fluids, fusion, populations

Theme	Mathematics	Representative domains
Uncertainty	Probability, statistics, Bayesian methods, information theory	Sensors, experiments, climate, robotics
Optimisation	Convex, combinatorial and variational optimisation	Chip design, antennas, protein folding, logistics
Learning	ML, deep learning, foundation models, surrogates	Vision, language, scientific AI, materials
Control	Control theory, feedback, reinforcement learning	Robotics, autonomous vehicles, grids, manufacturing
Computation	Complexity, numerical methods, HPC, quantum computing	Simulation, cryptography, quantum algorithms

4. From Themes to Verbs

The seven themes are nouns: they name the questions a field poses. But a working Deep Tech system does not merely pose questions — it performs operations. Recast as actions, the themes become seven mathematical verbs: Represent, Simulate, Infer, Optimise, Learn, Control and Compute. The themes and the verbs are two views of the same structure, the first conceptual and the second operational.

Theme (the question)	Verb (the action)	Operation
Representation	Represent	Encode the system as a mathematical object
Dynamics	Simulate	Evolve the state forward in time
Uncertainty	Infer	Update beliefs from incomplete, noisy data
Optimisation	Optimise	Search for the best configuration or decision
Learning	Learn	Approximate an unknown function from data
Control	Control	Close the loop to steer behaviour toward a goal
Computation	Compute	Determine what is feasible, and at what cost

The central hypothesis follows: **Can every Deep Tech company be seen as some combination of these verbs?**

This is easiest to see by decomposing three domains that appear to have nothing in common.

	Fusion (confinement)	Drug discovery	Autonomous vehicle
Represent	Plasma field configuration	Molecular graph / conformation	Vehicle + environment state
Simulate	Magnetohydrodynamic evolution	Molecular dynamics	Vehicle dynamics + sensor model

	Fusion (confinement)	Drug discovery	Autonomous vehicle
Infer	Reconstruct state from diagnostics	Binding affinity under noise	Localise position under sensor noise
Optimise	Maximise confinement time	Maximise binding and drug-likeness	Minimise-cost trajectory
Learn	Surrogate for plasma transport	Learned structure prediction	Perception and prediction networks
Control	Real-time coil and field control	(less central)	Closed-loop driving policy
Compute	HPC plasma codes	Docking and free-energy at scale	On-board real-time inference

5. The Modelling Stack: Reality → Equation → Simulation → Surrogate

For most of the modern scientific era the pipeline ran in three stages.

Reality → Equation → Simulation

One observes a phenomenon, writes down governing equations — Navier–Stokes for fluids, the Schrödinger equation for quantum systems, the equations of magnetohydrodynamics for plasma — and then solves them numerically because closed-form solutions rarely exist.

A closed-form solution is one that can be written down explicitly as a finite mathematical expression, whereas a numerical solution is obtained through a computational procedure that approximates the answer.

In this sense, simulation becomes significant precisely because the accurate closed form representations do not exist.

However, this raises a new problem. High-fidelity simulation is frequently the bottleneck: a single computational fluid dynamics run, a molecular dynamics trajectory or a plasma code can consume enormous compute and still be too slow to sit inside a design loop. The AI driven emerging pattern adds a fourth stage.

Reality → Equation → Simulation → Learned Surrogate

A learned surrogate is a model trained to approximate the expensive simulation at a fraction of its cost. The pattern is now visible across the frontier: computational fluid dynamics gives way to neural CFD; physics-based protein folding is superseded by AlphaFold's learned map from sequence to structure; classically designed controllers are replaced by learned policies. In each case, learning is not a separate consumer-AI technique bolted onto science — it has become a computational method for doing science.

This convergence is the reason the curriculum gives learning two distinct modules rather than one. The themes *Learning* and *Computation* are merging: a trained surrogate is simultaneously an act of learning and a strategy for computation. Treating learned models as

first-class citizens of the modelling stack, rather than as an afterthought, is one of the defining commitments of the approach proposed here.

6. Proposed Curriculum: Ten Modules

The seven themes expand naturally into ten modules.

Module 1 — How We Model Reality

Core question. *How do we represent a physical system mathematically?*

Topics. State spaces, vector spaces, tensors, coordinate systems, geometry and dimensionality.

Applications. Robots, molecules, quantum states and climate models.

Outcome. Students understand what a “model” actually is — a choice of representation, not a fixed feature of the world.

Module 2 — How Systems Change

Core question. *How do systems evolve over time?*

Topics. Calculus, ordinary and partial differential equations, dynamical systems, chaos and stability.

Applications. Fusion, aerospace, energy grids and weather.

Outcome. Students learn the mathematics of dynamics, and why stability is as important as the trajectory itself.

Module 3 — How We Simulate Nature

Core question. *What do we do when the equations cannot be solved?*

Topics. Numerical methods, finite differences, finite elements, Monte Carlo and scientific computing.

Applications. Computational fluid dynamics, materials, drug design and plasma simulation.

Outcome. Students learn why simulation became a trillion-dollar industry — and where its limits lie.

Module 4 — How We Reason Under Uncertainty

Core question. *How do we make decisions when information is incomplete?*

Topics. Probability, statistics, Bayesian inference and uncertainty quantification.

Applications. Sensors, robotics, medical diagnosis and risk analysis.

Outcome. Students learn to treat uncertainty as a first-class engineering object rather than as noise to be ignored.

Module 5 — How We Optimise

Core question. *How do we find the best solution?*

Topics. Convex and non-convex optimisation, variational methods and control optimisation.

Applications. Chip design, logistics, antenna design and protein folding.

Outcome. Students learn that a great deal of engineering is, structurally, optimisation.

Module 6 — How We Learn Models

Core question. *What if the equations are unknown?*

Topics. Statistical learning, deep learning, representation learning and foundation models.

Applications. Vision, language, scientific AI and autonomous systems.

Outcome. Students understand learning as the approximation of functions from data under constraints.

Module 7 — How We Learn Physics

Core question. *Can AI discover and accelerate physical law?*

Topics. Physics-informed learning, neural ODEs, scientific machine learning and surrogate models.

Applications. Materials, climate, fusion and drug discovery.

Outcome. Students see exactly where AI meets science — the learned-surrogate layer of the modelling stack.

Module 8 — How We Control Complex Systems

Core question. *How do we make systems do what we want?*

Topics. Control theory, reinforcement learning, feedback, observability and controllability.

Applications. Robotics, autonomous vehicles, manufacturing and energy systems.

Outcome. Students learn the mathematics of action — closing the loop between observation and intervention.

Module 9 — How Information Flows

Core question. *What are the limits of communication and computation?*

Topics. Information theory, coding theory, entropy and complexity theory.

Applications. Telecoms, quantum, AI and cybersecurity.

Outcome. Students understand the fundamental limits that no engineering effort can cross.

Module 10 — How Nature Computes

Core question. *Can physics itself perform computation?*

Topics. Quantum computing, photonic computing and neuromorphic computing.

Applications. Quantum AI, photonic AI and next-generation hardware.

Outcome. Students come to see computation as a physical process, not an abstraction running above the world.

How the modules map to themes and verbs

Module	Theme / Verb	Place in the stack
1 Model Reality	Representation / Represent	Encode the state
2 Systems Change	Dynamics / Simulate	Equation
3 Simulate Nature	Computation / Simulate	Simulation
4 Reason Under Uncertainty	Uncertainty / Infer	Beliefs over states

Module	Theme / Verb	Place in the stack
5 Optimise	Optimisation / Optimise	Search the design space
6 Learn Models	Learning / Learn	General surrogates
7 Learn Physics	Learning $\cap$ Dynamics / Learn	Learned surrogate
8 Control Complex Systems	Control / Control	Close the loop
9 Information Flows	Computation / Compute	Fundamental limits
10 Nature Computes	Computation / Compute	Physical computation

7. Pedagogical Principles

The argument implies a small number of concrete commitments that distinguish this approach from a conventional applied-mathematics course.

- **Problem-first sequencing.** Every module opens with a frontier problem that creates genuine demand for the mathematics, rather than presenting the mathematics and searching for an application afterwards.
- **One structure, many domains.** Recurrence is taught explicitly: when eigenstructure, stability or function approximation reappears in a new domain, the connection is named, not left for the student to discover by accident.
- **Simulation literacy.** Numerical methods are treated as a core competence alongside derivation, because most equations of interest are met only through simulation.
- **Surrogate awareness.** Learned models are introduced as a legitimate layer of the modelling stack — a way of computing — rather than as a separate AI topic.
- **Uncertainty as a first-class object.** Distributions, not point estimates, are the default description of any measured quantity.
- **Computation as physical.** Feasibility and hardware — including emerging quantum, photonic and neuromorphic substrates — are treated as part of the mathematics, not as an implementation detail beneath it.
- **Expansive scope:** There are new techniques to expand the meaning of ‘surrogate’ as we discuss below

8. Expansion of the idea of the Learned Surrogate

Finally, we can expand the idea of the Learned Surrogate.

We defined this idea of the Learned Surrogate as

A surrogate is a computational model that stands in for a more expensive, inaccessible, complex, or unknown process while preserving the behaviour that matters for a given task.

Which translates to the progression: Reality $\rightarrow$ Equation $\rightarrow$ Simulation $\rightarrow$ Surrogate

One could argue that the last decade of AI has been about creating increasingly powerful forms of surrogates - and thus, we can expand the idea of the surrogate to:

A surrogate is a learned representation that substitutes for some aspect of reality, computation, reasoning, planning, or decision-making.

This leads to several different classes of surrogates for example:

Function Surrogates: This is the classic interpretation as we noted above. Reality $y=f(x)$ but f is expensive.- so we learn an analogous function $\hat{f}(x)$ - $\hat{f}$ hat x

Representation Surrogates: Here we do not replace a simulation. We replace the representation itself. Examples: Word embeddings instead of dictionary definitions.

World Models: This is where things become very interesting. A world model is essentially a surrogate for the dynamics of the environment. Instead of: Reality $\rightarrow$ Equation $\rightarrow$ Simulation - we learn: Reality $\rightarrow$ World Model. The world model becomes a learned simulator. The world model becomes a surrogate for the dynamics.

Reinforcement Learning as Surrogate Search: Recent work by [Prof David Silver and Richard Sutton](#) expands the idea of Reinforcement learning in the direction of surrogate search. Traditionally: Control theory requires Dynamics and a Cost Function to be known.

RL replaces explicit control design with learning. A policy is a surrogate for optimal control. In the classical view, the Engineer derives a controller. In the RL View, the Agent learns the controller. Policy network becomes a surrogate controller.

Value Functions as Surrogates: Here, the value function is itself the surrogate. Instead of exploring all futures, the value function summarizes future outcomes. In this mode, value functions are surrogates for future consequences.

Planning Surrogates: Traditional search is an enormous tree. Learned search is value network and policy network - which effectively replaces much of the tree. The learned model acts as a surrogate for exhaustive search.

Scientific Discovery/ Candidate hypothesis Surrogates:

This model extends the ideas even further as: Physics $\rightarrow$ Surrogate $\rightarrow$ Candidate Discovery
The surrogate narrows the search space dramatically for possible feasible candidate hypotheses.

Cognitive Surrogates - LLM as a surrogate

LLMs introduce a new category. An LLM can act as a surrogate for - especially if fine tuned and trained for a specific function.

Agentic Surrogates:

An AI agent becomes a surrogate for a workflow. The surrogate is no longer replacing a function. It is replacing a process.

A Unified Hierarchy

One possible way to think of this is to see what the surrogate can replace.

Surrogate Type	Replaces
Function surrogate	Simulation
Representation surrogate	Feature engineering
World model	Dynamics
Policy surrogate	Controller
Value surrogate	Future outcomes
Planning surrogate	Search
Scientific surrogate	Experimentation
Cognitive surrogate	Human reasoning
Agentic surrogate	Human workflows

9. Conclusion

Deep Tech can look, from the outside, like an unrelated amalgamation of frontier industries, each demanding its own specialised mathematical training. The paper has argued the opposite: that beneath fusion, robotics, drug discovery, artificial intelligence, quantum, climate and semiconductors lies a compact and shared mathematical structure — seven themes, expressed as seven verbs, recurring with remarkable regularity. Every Deep Tech company is some combination of representing, simulating, inferring, optimising, learning, controlling and computing.

These theories are expanding wit ideas like world models and reinforcement learning.

That observation is both a teaching device and a lens. As a teaching device, it justifies inverting the conventional curriculum: teaching mathematics through Deep Tech rather than before it, so that each abstraction is earned against a problem the learner already cares about.

As a lens, it gives founders, investors and engineers a way to read any frontier company in terms of the mathematical operations it actually performs, and to recognise the same structure beneath very different physics. The ten-module curriculum set out above is one concrete realisation of that lens — an attempt to give a practitioner the smallest set of mathematical structures that makes the frontier legible.

A lot is at stake here - in terms of national competitiveness, jobs, skills etc.

I want to approach this in an open manner with the idea of creating an ecosystem - especially in the UK and Europe.

[Ajit Jaokar](#)
[Erdos Research](#)